

Les processus

Cours n°3
Administration Système – ESIEE- E3INFO
2025/2026

Clément BOIN
clement.boin@esiee.fr

Plan

- Introduction sur les processus
- Gestion des processus sous Linux
 - principales commandes
 - les signaux
 - les modes d'exécution
 - Les redirections

Introduction

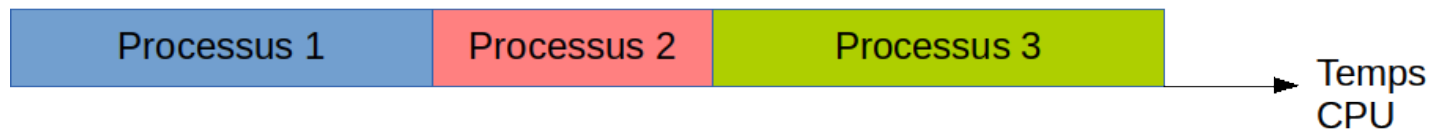
Les processus : définitions

- Un processus est un **programme en exécution**.
 - Un espace mémoire
 - Des ressources utilisées (fichiers, périphériques...)
- Plusieurs processus peuvent être exécutés **en même temps**.

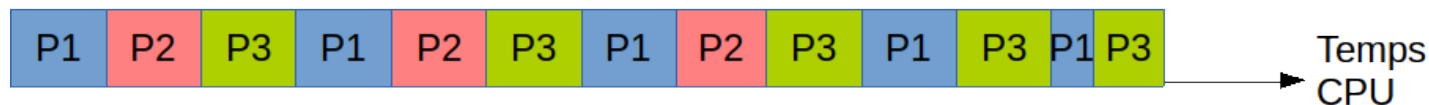
Monotâche/Multitâche

- Monotâche :
 - un seul processus à la fois
 - Pas de problème de gestion de la mémoire
 - Pas de conflit de ressources

Monotâche



Multitâche



- Multitâche
 - Mise en commun, partage
 - Attention à la gestion, aux conflits et à la sécurité

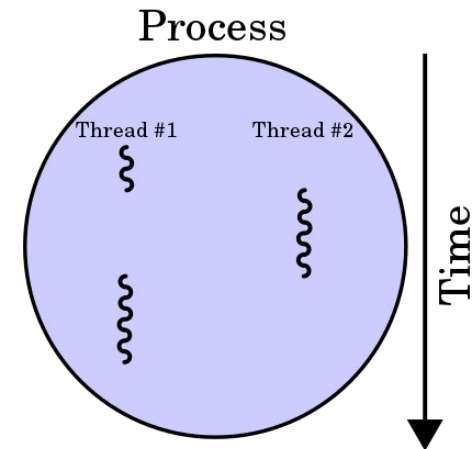
Différents multitâches

Petit historique

- À l'origine (années 1960) : **Temps partagé**
 - Au bon vouloir du programme
 - Aucune résistance aux erreurs, et aux malveillances
- **Coopératif** (jusqu'à Windows 3.11)
 - Forme simple de multitâche
 - Chaque tâche permet explicitement aux autres tâches de s'exécuter
 - Risque fort de blocage si un processus plante
- **Préemptif** (sur tous les OS modernes)
 - Les processus sont interrompus si trop longs
 - Un 'ordonnanceur' gère le déroulement
 - Il est prioritaire (sinon ça ne marche pas)

Processus vs Thread

- **Le flot de contrôle** représente :
 - Déroulement du programme,
 - Progression dans la liste des instructions
 - Raisonnement à l'œuvre (algorithme)
- **Processus** : un seul flot de contrôle
- Il existe une plus petite unité d'exécution appelée **thread** (*fil*):
 - plusieurs flots de contrôle dans le même processus
 - utile pour gérer de multiples utilisateurs, ou améliorer l'utilisation d'une machine multiprocesseurs...
 - des conflits sont possibles, contrairement au processus



Les processus

Qu'est-ce qu'un processus ?

- Un ensemble d'instructions
- Une zone mémoire définie et bornée
- Des méta informations
 - Propriétaire
 - Temps processeur consommé
 - Fichiers ouverts
 - etc

Organisation (du côté du processus)

- Le processus se considère **seul au monde** :
 - pas à se soucier de la gestion de l'ensemble
- Le processus a accès à un **espace mémoire continu, cohérent**
 - Les variables du processus sont protégées, uniques, fiables
- Les **conflits d'accès sont délégués** au système
 - Le processus demande l'accès aux ressources (via l'OS) : il sera prévenu quand le résultat est disponible (mis en attente pendant ce laps de temps)

Organisation (du coté de l'OS)

- L'OS doit **tout savoir**, sur tous les processus
 - priorité,
 - propriétaire,
 - ressources manipulées,
 - temps consommé
 - ...
- L'OS vérifie la **sécurité**, et bloque les tentatives illicites
- L'OS **connaît tout le monde**, et ment à tout le monde
 - il **choisit** qui travaille,
 - et **interrompt** celui qui travaille depuis trop longtemps
 - il donne de la **mémoire** selon les dispos, et fait croire que tout est contiguë
 - il donne **accès aux ressources**, selon ses droits, et les disponibilités
 - il **endort** le processus demandeur, et le **réveille** lorsque les réponses arrivent

Gestion de processus (sous Linux)

Arborescence des processus (sous Linux)

- Au démarrage du système un processus nommé **systemd** est lancé (init sur certains linux comme Ubuntu)
 - responsable du lancement des autres processus
 - hiérarchie de processus de racine systemd
- Chaque processus est identifié par :
 - un **numéro unique** (PID : process identifier),
 - l'identité du **propriétaire**
 - le numéro du **processus père** (PPID) celui à partir duquel il a été lancé.
 - Le PID 1 est celui de systemd

Commande pstree

Hiérarchie des processus

```
clement@boko:~$ pstree
systemd--ModemManager--2*[{ModemManager}]
      |
      |--NetworkManager--2*[dhclient]
      |                   |
      |                   |--2*[{NetworkManager}]
      |
      |--accounts-daemon--2*[{accounts-daemon}]
      |
      |--acpid
      |
      |--apache2--5*[apache2]
      |
      |--avahi-daemon--avahi-daemon
      |
      |--boltd--2*[{boltd}]
      |
      |--colord--2*[{colord}]
      |
      |--cron
      |
      |--cups-browsed--2*[{cups-browsed}]
      |
      |--cupsd--dbus
      |
      |--dbus-daemon
      |
      |--dropbox--86*[{dropbox}]
      |
      |--firefox--Web Content--48*[{Web Content}]
      |           |
      |           |--Web Content--37*[{Web Content}]
      |           |
      |           |--Web Content--24*[{Web Content}]
      |           |
      |           |--WebExtensions--32*[{WebExtensions}]
      |           |
      |           |--80*[{firefox}]
      |
      |--fwupd--4*[{fwupd}]
      |
      |--gdm3--gdm-session-wor--gdm-wayland-ses--gnome-session-b--gnome-sh+
      |           |           |           |           |
      |           |           |           |           |--gsd-a11y+
      |           |           |           |           |--gsd-clip+
```

Sur une machine cliente (extrait)

```
test@ubuntu:~$ pstree
systemd--accounts-daemon--{gdbus}
      |                   |
      |                   |--{gmain}
      |
      |--acpid
      |
      |--apache2--5*[apache2]
      |
      |--atd
      |
      |--cron
      |
      |--dbus-daemon
      |
      |--dhclient
      |
      |--2*[iscsid]
      |
      |--login--bash--pstree
      |
      |--lvm2metad
      |
      |--lxcfs--2*[{lxcfs}]
      |
      |--mdadm
      |
      |--mysqld--26*[{mysqld}]
      |
      |--polkitd--{gdbus}
      |           |
      |           |--{gmain}
      |
      |--rsyslogd--{in:imklog}
      |           |
      |           |--{in:imuxsock}
      |           |
      |           |--{rs:main Q:Reg}
      |
      |--snapd--6*[{snapd}]
      |
      |--systemd--(sd-pam)
      |
      |--systemd-journal
      |
      |--systemd-logind
      |
      |--systemd-timesyn--{sd-resolve}
      |
      |--systemd-udev
```

Sur un serveur
sans interface graphique
(tous les processus)

Le PID

Process Identifier

- Unique à l'instant T, propre à chaque processus
- 0 c'est le système, 1 c'est init (systemd)
- Accessible en langage shell (bash) avec \$\$
 - `echo $$`
- Chaque processus a un père
 - Accessible avec `$PPID`
- Chaque processus peut avoir de 0 à n fils

Quelques commandes système liées à la gestion des processus

- `top` (et autre variantes : `htop`, `atop`)
- `ps`
- `pstree`

- `free` : état de la mémoire vive

- `iostat -h` : état du processeur
(à installer : `apt install sysstat`)

- `nohup` : exécuter une commande résistante aux déconnexions

- `nice` : modifier la priorité d'une commande

La commande ps

- Permet de **connaître** toutes les informations des **processus actifs à un moment donné** :
 - Chaque processus est identifié par un nombre unique (PID).
 - TTY : à quel port de terminal est associé le processus.
 - STAT : état du processus + indicateurs
 - S comme "sleep" : endormi
 - R comme "run" : en cours d'exécution
 - TIME : depuis combien de temps le processus utilise les ressources du microprocesseur.
 - COMMAND précise la commande

```
test@ubuntu:~$ ps au
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1034	0.0	0.3	65832	3400	tty1	Ss	22:45	0:00	/bin/login --
test	1294	0.0	0.5	22916	5508	tty1	S	22:46	0:00	-bash
test	1427	0.0	0.3	37472	3412	tty1	R+	23:40	0:00	ps au

- `ps aux` (syntaxe BSD)
 - permet de connaître les utilisateurs associés à chaque processus (en incluant les applications du serveur graphique X)
- `ps axms`
 - permet d'avoir des informations sur les threads

Détails d'un processus

- `/proc/<PID>/` est un répertoire qui contient toutes les informations sur le processus :
 - `cmdline` : texte de la ligne de commande ayant lancé le processus
 - `exe` : lien vers le fichier exécutable
 - `environ` : contenu de l'environnement
 - `fd` : liens vers les fichiers (descripteurs) ouverts
 - 1 : stdin (entrée standard)
 - 2 : stdout (sortie standard)
 - 3: stderr (sortie d'erreur)
 - ...

- Exemple

```
clement@CAMANA2:~$ ps
  PID TTY          TIME CMD
   12 pts/0        00:00:00 bash
  200 pts/0        00:00:00 ps
clement@CAMANA2:~$ ls /proc/12/fd
0 1 2 255
clement@CAMANA2:~$ ls -l /proc/12/fd/255
lrwx----- 1 clement clement 64 Oct  1 13:57 /proc/12/fd/255 -> /dev/pts/0
clement@CAMANA2:~$ ls /proc/200/
ls: cannot access '/proc/200/': No such file or directory
```

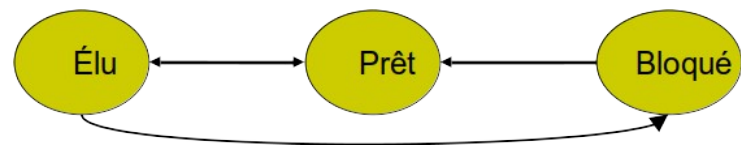
Etat d'un processus

- Chaque processus possède son propre **environnement d'exécution**.
- 3 états principaux possibles :
 - R - en exécution (élu),
 - S - en attente d'exécution (prêt) : tout sauf le processeur
 - T – arrêté (bloqué) : attend une ressource non encore disponible (attente de saisie d'une valeur)

```
clement@CAMANA2:~$ ps -o pid,comm,state,stat,user
PID COMMAND      S  STAT USER
 12  bash          S  Ss   clement
 30  nano          T  T    clement
 36  ps            R  R+   clement
```

+ : Au premier plan

s : Leader d'un groupe de sessions



La commande **top**

- Permet d'afficher des **informations en continu** sur l'activité du système.
 - ressources que les processus utilisent
 - **quantité** de RAM
 - **pourcentage** de CPU
 - la **durée** de ce processus depuis son démarrage
- On peut par exemple **stopper un processus**
 - taper **k**.
 - Saisir le PID du processus
 - Saisir le signal de fin de processus : 15 (SIGTERM)
 - 9 (SIGKILL) est plus brutal !
- Pour quitter top, appuyer sur la touche "**q**".

Les signaux

Communication inter-processus

- Les processus peuvent **communiquer entre eux** par l'envoi de messages appelés signaux.
 - Asynchrone
 - Nombre limité
 - Réactions prédéfinies
- **Arrêter un processus (interrompre)**
 - signal SIGINT
 - signal numéro 2
 - généré depuis le clavier en appuyant simultanément sur les touches CTRL+C.
- **Le signal de terminaison**
 - SIGKILL
 - signal numéro 9
 - ne peut pas être inhibé

La commande **kill**

- Permet d'expédier un signal à un processus en cours.
`kill [options] PID`
- Si vous n'arrivez pas à tuer un processus, vous devez utiliser la commande :
`kill -9 PID`
- `killall` permet aussi de tuer un processus en indiquant son nom.

Commande trap -l

Liste des signaux

```
clement@boko:~$ trap -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS      8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM    15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT    19) SIGSTOP    20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH   29) SIGIO      30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

- 64 messages différents
 - USR1/USR2 : Signaux personnalisés utilisateurs
- Interruption du flot de contrôle d'un processus
- Proviennent d'un événement matériel ou logiciel

Exemple de notifications entre processus

- Les signaux permettent à un processus de **notifier ou d'interrompre** un autre processus
- La commande interne **trap** permet de capturer ces signaux

Exemple de script tran ch :

```
trap 'echo "Interruption !"' SIGUSR1
```

```
while true; do
    echo "Travail en cours..."
    sleep 1
done
```

```
clement@CAMANA2:~$ ./trap.sh
```

```
Travail en cours...
Travail en cours...
Travail en cours...
Travail en cours...
Travail en cours...
Travail en cours...
Travail en cours...
Travail en cours...
Interruption !
Travail en cours...
Travail en cours...
Travail en cours...
Interruption !
Travail en cours...
Travail en cours...
Travail en cours...
Travail en cours...
Travail en cours...
Travail en cours...
clement@CAMANA2:~$
```

1^{er} terminal :
exécution du
script

```
PID TTY TIME CMD
3090 pts/1 00:00:00 bash
3374 pts/1 00:00:00 ps
clement@CAMANA2:~$ ps a
PID TTY STAT TIME COMMAND
  1 hvc0 S1+ 0:00 /init
  7 hvc0 S1+ 0:00 plan9 --control-sock
et 6 --log-level 4 --server- 3090 pts/1 Ss
0:00 -bash
3184 pts/2 Ss 0:00 -bash
3369 pts/2 S+ 0:00 -bash
3379 pts/2 S+ 0:00 sleep 3
3381 pts/1 R+ 0:00 ps a
clement@CAMANA2:~$ kill -SIGUSR1 3369
clement@CAMANA2:~$ kill -SIGUSR1 3369
clement@CAMANA2:~$
```

2^{ème} terminal :
envoi du signal

Les modes d'exécution

Mode d'exécution séquentiel

Foreground (premier plan)

On tape les commandes les unes après les autres

- Entrée entre chaque commande

```
$sleep 5
```

```
pwd
```

```
$pwd
```

```
/home/uti
```

Apparition au bout de
5 secondes

- On peut également séparer les commandes par des ;

```
$sleep 5;pwd;uname
```

```
/home/uti
```

```
Linux
```

- Pour arrêter une exécution on tape Ctrl-C

Exécution en arrière plan (1/2)

Background

- Opérateur &
- Permet de lancer une commande et récupérer la main immédiatement

```
cboin@clement-boin:~$ sleep 5&
[1] 1085
cboin@clement-boin:~$ ps
  PID TTY          TIME CMD
  986 tty1        00:00:00 bash
 1085 tty1        00:00:00 sleep
 1086 tty1        00:00:00 ps
cboin@clement-boin:~$ ps
  PID TTY          TIME CMD
  986 tty1        00:00:00 bash
 1085 tty1        00:00:00 sleep
 1087 tty1        00:00:00 ps
cboin@clement-boin:~$ ps
  PID TTY          TIME CMD
  986 tty1        00:00:00 bash
 1088 tty1        00:00:00 ps
[1]+  Done                  sleep 5
cboin@clement-boin:~$ ps_
```

Exécution en arrière plan (2/2)

- On peut interagir avec les processus avec des commandes spécifiques :
 - **Ctrl-Z** : Arrête la tâche au premier plan pour la mettre en arrière plan
 - **jobs** : Liste les travaux en cours
 - **bg** : place la tâche actuelle en arrière-plan
 - **fg** : place une tâche actuelle en arrière-plan

```
clement@CAMANA2:test$ ping www.esiee.fr > /dev/null &
[1] 136
clement@CAMANA2:test$ sleep 100&
[2] 137
clement@CAMANA2:test$ jobs
[1]-  Running                  ping www.esiee.fr > /dev/null &
[2]+  Running                  sleep 100 &
clement@CAMANA2:test$ fg
sleep 100
^C
clement@CAMANA2:test$ jobs
[1]+  Running                  ping www.esiee.fr > /dev/null &
clement@CAMANA2:test$
```

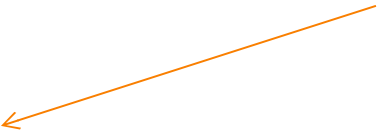
La commande **wait** (1/2)

- Commande interne du shell, qui permet d'**attendre la fin** de ses processus **enfants**
- Deux formes possibles :
 - `wait PID` : attend ce PID précisément.
 - Le code de sortie (= valeur renvoyée par `exit`) du fils décédé est récupérable dans la variable spéciale `$?`
 - `wait` : attend tous les fils

La commande **wait** (2/2)

```
$ gedit &  
[3] 10789  
$ echo $!  
10789  
$ wait 10789  
[3]+  Fini      gedit  
$ echo $?  
0  
$ gedit & emacs &  
[1] 11117  
[2] 11118  
$ wait  
[1]-  Fini      gedit  
[2]+  Fini      emacs  
$ echo $?  
0  
$
```

Validation au terminal (entrée)
puis fermeture de la fenêtre
gedit



Fermeture de gedit et emacs
(peu importe l'ordre)



Commandes composées

Si C1 et C2 sont deux *commandes simples*, alors:

- $C1 \ \&\& \ C2$: lance C1 puis C2 seulement si C1 a réussi
- $C1 \ || \ C2$: lance C1 puis C2 seulement si C1 a échoué
- $C1 \ ; \ C2$: lance C1 puis C2
- $C1 \ \&$: lance C1 **en arrière-plan** → exécution parallèle
- $C1 \ | \ C2$: lance C1 et C2 en parallèle et en redirigeant la sortie standard de C1 vers l'entrée standard de C2 (**tube**)

Priorités des opérateurs de contrôle :

- Une ligne de commande est :
 - **simple** si elle ne comporte aucun opérateur de contrôle ;
 - **composée** dans le cas contraire.
- On applique les règles de priorités suivantes:

Priorités égales (*)



()

{ }

|

&& ||

& ;

::



(*) L'opérateur rencontré le premier de gauche à droite remporte la priorité

Priorités décroissantes
(priment sur les priorités
horizontales)

Les redirections

Processus et redirections

- A son lancement, tout processus dispose de **trois canaux** ouverts par le système :
 - `stdin`, `stdout`, `stderr`
- A ces canaux sont associés des **fichiers par défaut** :
 - les caractères tapés dans le terminal d'exécution alimentent `stdin`
 - Le terminal est alimenté par `stdout` et `stderr`
- Les **redirections** permettent d'associer **d'autres fichiers** aux canaux que ceux par défaut.

```
$ ls
```

```
cap1.tiff          cap2.tiff          test
```

```
$ ls > /tmp/toto # redirige stdout vers /tmp/toto
```

```
$ ls -l /tmp/toto
```

```
-rw-r--r--    1 clement clement 25 Feb 25 23:24 /tmp/toto
```

Toutes les redirections

`[n]> fichier` Redirige stdout (ou n) vers le fichier

`[n]>> fichier` Ajoute stdout (ou n) en fin de fichier

`[n]< fichier` Alimente stdin (ou n) à partir du fichier

`[n1]>&n2` Unifie stdout (ou n1) au canal n2 en sortie

`[n1]<&n2` Unifie n2 et stdin (ou n1) en entrée

`[n]<> fichier` Associer le fichier en lecture/écriture à stdin (ou n)

`[n]<<chaîne` Lit sur stdin (ou n) jusqu'à l'apparition de la chaîne

`&> fichier` Redirige stdout et stderr vers le fichier

`&>> fichier` Ajoute stdout et stderr en fin de fichier

Exemple

Analyse d'une ligne de commande

```
$ test -d    '/sw'    2>/dev/null && cat  
$FIC | wc -l || echo  "/sw inexistant"
```

- Distinguer les mots des opérateurs.

```
$ test -d '/sw' 2> /dev/null && cat  
$FIC | wc -l || echo  "/sw inexistant"
```

- Cette ligne vérifie que /sw est un dossier, puis calcule le nombre de ligne du fichier \$FIC, sinon affiche un message d'erreur.